

Aula 06 — Árvores de Regressão e *Ensembles*

Curso de Aprendizado de Máquina — UFF

Prof. Gabriel Sanfins

Leitura recomendada: AME §4.8–4.10; ISLP cap. 8

O que você vai aprender

Ao final desta aula você deve ser capaz de:

- descrever como uma **árvore de regressão** particiona o espaço de covariáveis e prediz por médias locais;
- explicar a construção em duas etapas: *crescimento* por divisões binárias recursivas e *poda* por custo-complexidade;
- justificar, via redução de variância, por que **agregar** estimadores ajuda;
- descrever **bagging**, a estimativa *out-of-bag* e a medida de importância de variáveis;
- explicar como as **florestas aleatórias** *descorrelacionam* as árvores e o papel de m ;
- descrever **boosting** como ajuste incremental dos resíduos e o papel de λ , B e da profundidade.

Árvores são o método mais *interpretável* que veremos — e, sozinhas, das piores em predição. A história desta aula é como transformá-las, por agregação, em alguns dos métodos *mais poderosos* que existem para dados tabulares (florestas, XGBoost). A pergunta que organiza tudo:

como combinar muitos modelos ruins para obter um ótimo?

A resposta tem nome e sobrenome — Proposição 2.1 — e duas condições. Cada método desta aula é uma tentativa de satisfazer melhor a segunda delas.

1 Árvores de regressão

Uma árvore parte o espaço de covariáveis em regiões disjuntas $R_1, \dots, R_J$ por *particionamento recursivo* e prediz, em cada região, a média das respostas de treino ali contidas:

$$g(\mathbf{x}) = \frac{1}{|\{i : \mathbf{X}_i \in R_k\}|} \sum_{i: \mathbf{X}_i \in R_k} Y_i, \quad \text{para } \mathbf{x} \in R_k. \quad (1)$$

Para prever, percorre-se a árvore do topo às folhas seguindo as condições (“ $x_j < t$?”) em cada nó.

Ideia central

Vantagens estruturais das árvores: (i) altamente **interpretáveis**; (ii) lidam *nativamente* com covariáveis discretas (cortes do tipo $x_j \in \{\text{verde}, \text{vermelho}\}$ — sem precisar criar *dummies*); (iii) fazem **seleção de variáveis** automática; (iv) capturam **interações** sem que precisemos especificá-las.

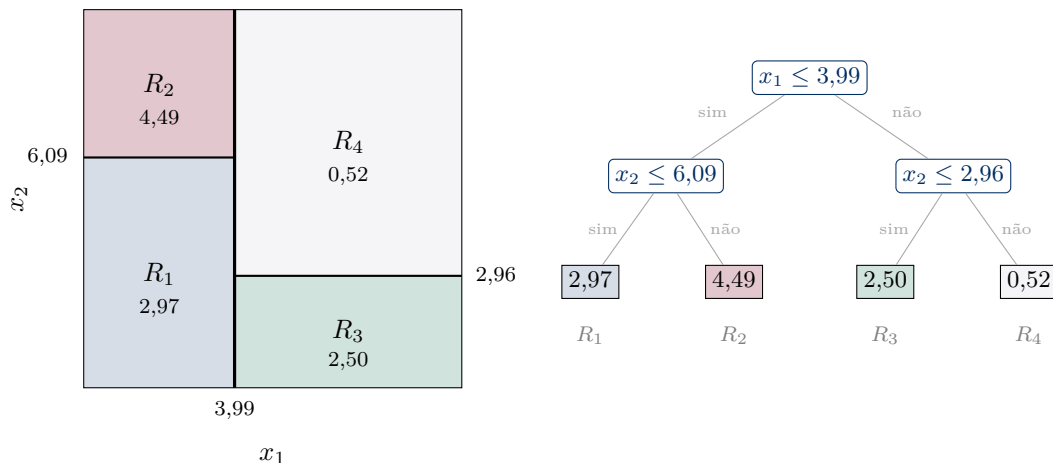


Figura 1: Os dois lados da mesma coisa. À esquerda, a partição de $[0, 10]^2$ em quatro retângulos; à direita, a árvore que a produz. Cada nó interno é um corte perpendicular a um eixo, cada folha é uma região, e o número na folha é a média (1) das respostas de treino ali. Os valores são de um ajuste real ($n = 300$, profundidade 2, $R^2 = 0,941$ no treino). Repare que o corte de x_2 é *diferente* nos dois ramos (6,09 à esquerda, 2,96 à direita): é assim que a árvore captura uma interação sem que ninguém a tenha especificado.

Atenção: e a desvantagem estrutural

Os cortes são sempre *perpendiculares aos eixos*. Uma fronteira diagonal simples, como $x_1 + x_2 > 10$, obriga a árvore a aproximá-la por uma escadinha de muitos cortes — e cada degrau custa dados. Se você suspeita de relações lineares nas covariáveis, um modelo linear (Aula 02) faz melhor com muito menos.

1.1 Etapa 1 — crescimento (divisões binárias recursivas)

Idealmente escolheríamos a partição que minimiza o EQM $P(T) = \sum_R \sum_{i: \mathbf{X}_i \in R} (Y_i - \hat{y}_R)^2 / n$. Mas buscar a árvore ótima é computacionalmente inviável (NP-difícil). Usa-se então uma heurística *gananciosa*: a cada passo, escolhe-se a covariável x_j e o corte t que mais reduzem a soma de quadrados ao dividir uma região em $R_1 = \{x_j < t\}$ e $R_2 = \{x_j \geq t\}$:

$$\min_{j, t} \sum_{i: \mathbf{X}_i \in R_1} (Y_i - \hat{y}_{R_1})^2 + \sum_{i: \mathbf{X}_i \in R_2} (Y_i - \hat{y}_{R_2})^2. \quad (2)$$

Repete-se recursivamente em cada região-filha, até um critério de parada (ex.: menos de 5 observações por folha). Isso gera uma árvore grande T_0 , que ajusta bem o treino mas tende a **superajustar**.

Observação 1.1. “Ganancioso” quer dizer que a escolha de (2) olha *um passo* à frente e nunca volta atrás: o primeiro corte é o que mais reduz o RSS agora, ainda que outro corte pior agora permitisse cortes muito melhores depois. Não é um defeito de implementação — é o preço de fugir de um problema NP-difícil. Guarde isso: parte do ganho dos *ensembles* vem justamente de compensar essa miopia.

1.2 Etapa 2 — poda por custo-complexidade

Para reduzir a variância, *podamos* T_0 . A poda por custo-complexidade introduz um parâmetro $\alpha \geq 0$ e busca a subárvore $T \subseteq T_0$ que minimiza

$$\sum_{k=1}^{|T|} \sum_{i: \mathbf{X}_i \in R_k} (Y_i - \hat{y}_{R_k})^2 + \alpha |T|, \quad (3)$$

onde $|T|$ é o número de folhas. O termo $\alpha |T|$ penaliza árvores grandes — é exatamente a mesma estrutura da regularização da Aula 02, com $|T|$ no papel de $\|\beta\|_1$: ajuste + $\lambda \times$ complexidade. α é o **tuning parameter**, escolhido por **validação cruzada** (Aula 03): $\alpha = 0$ devolve T_0 ; $\alpha \rightarrow \infty$ colapsa para a raiz (média global).

Atenção

Crescer e podar usam objetivos diferentes *de propósito*: cresce-se de forma gananciosa (olhando só um passo à frente) e corrige-se globalmente na poda. Não confunda profundidade máxima (parâmetro de crescimento) com α (parâmetro de poda) — ambos controlam complexidade, mas em etapas distintas.

2 Por que agregar? Redução de variância

Árvores isoladas têm *alta variância*: pequenas mudanças nos dados mudam bastante a árvore — basta que o corte da raiz mude um pouco para que toda a estrutura abaixo dele seja outra. A ideia dos *ensembles* é **combinar** muitas funções para reduzir essa variância.

Proposição 2.1 (Média de preditores reduz o risco). *Sejam $g_1, \dots, g_B$ estimadores não-viesados ($\mathbb{E}[g_b(\mathbf{x})] = r(\mathbf{x})$), de mesma variância $v(\mathbf{x}) = \text{Var}(g_b(\mathbf{x}))$ e não-correlacionados. Então a média $\bar{g} = \frac{1}{B} \sum_b g_b$ tem risco*

$$\mathbb{E}[(Y - \bar{g}(\mathbf{x}))^2 | \mathbf{x}] = \text{Var}(Y | \mathbf{x}) + \frac{v(\mathbf{x})}{B} \leq \mathbb{E}[(Y - g_b(\mathbf{x}))^2 | \mathbf{x}].$$

Demonstração. Pela decomposição viés-variância (Aula 01), o risco de qualquer preditor g é $\text{Var}(Y | \mathbf{x}) + \text{viés}^2 + \text{Var}(g(\mathbf{x}))$. Como cada g_b é não-viesado, $\bar{g}$ também é (viés = 0). Para a variância, sendo os g_b não-correlacionados e de variância $v(\mathbf{x})$, $\text{Var}(\bar{g}(\mathbf{x})) = \frac{1}{B^2} \sum_b \text{Var}(g_b(\mathbf{x})) = v(\mathbf{x})/B$. Substituindo obtém-se a igualdade; como $v(\mathbf{x})/B \leq v(\mathbf{x})$, o risco não aumenta. $\square$

Ideia central

Duas exigências da Prop. 2.1 guiam tudo o que segue: precisamos de preditores (i) **não-viesados** — por isso usamos árvores *não podadas* (profundas, baixo viés) — e (ii) **pouco correlacionados** — o ponto fraco, que as florestas aleatórias atacam diretamente.

Observação 2.2 (o que acontece quando a hipótese falha). A hipótese de não-correlação é forte demais para ser verdade, e vale saber o preço. Se as g_b têm correlação ρ duas a duas, a conta vira

$$\text{Var}(\bar{g}(\mathbf{x})) = \rho v(\mathbf{x}) + \frac{1-\rho}{B} v(\mathbf{x}) \xrightarrow{B \rightarrow \infty} \rho v(\mathbf{x}).$$

Ou seja: por mais árvores que você acrescente, a variância **não desce abaixo de $\rho v(\mathbf{x})$** . O termo que B mata é só o segundo. É por isso que reduzir ρ vale mais do que aumentar B — e é literalmente essa a ideia das florestas aleatórias.

3 Bagging

Bagging (*bootstrap aggregating*) gera diversidade reamostrando os dados. Para $b = 1, \dots, B$: sorteie uma amostra **bootstrap** (com reposição, tamanho n) e ajuste nela uma árvore *profunda* (não podada) g_b . A predição é a média:

$$g(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B g_b(\mathbf{x}). \quad (4)$$

Note a divisão de trabalho: as árvores são profundas (para ter viés baixo, condição (i)) e a média cuida da variância. Sozinha, cada árvore superajusta descaradamente — e é para isso que ela está ali.

3.1 Estimativa *out-of-bag* (OOB)

Cada amostra bootstrap omite, em média, $\approx 37\%$ das observações (as *out-of-bag*, Aula 03). Podemos prever cada $\mathbf{X}_i$ usando *só* as árvores que não a viram: isso dá uma estimativa de risco **de graça**, sem CV separada. Com B grande, cada observação fica de fora de umas $0,37B$ árvores — mais que suficiente para uma média confiável.

3.2 Importância de variáveis

Bagging sacrifica interpretabilidade: não há mais uma árvore para desenhar. Recupera-se parte dela com uma medida de **importância**: para cada variável, soma-se a *redução de RSS* em todas as divisões que a usaram, $RSS_{\text{pai}} - RSS_{\text{esq}} - RSS_{\text{dir}}$, e faz-se a média sobre as B árvores. Variáveis que reduzem muito o erro são as mais importantes.

Atenção

Essa importância é *enviesada* a favor de variáveis com muitos valores distintos (contínuas, ou categóricas com muitos níveis), simplesmente porque elas oferecem mais cortes candidatos e ganham por sorteio mais vezes. Ela também divide o crédito de forma arbitrária entre variáveis correlacionadas. Use-a para ordenar grosseiramente, não como evidência de causalidade nem para descartar variáveis com confiança.

4 Florestas aleatórias

O problema do *bagging*: as árvores ficam **correlacionadas**. Se há uma covariável muito forte, quase todas as árvores a usam no topo e ficam parecidas — e, pela observação acima, a variância trava em $\rho v(\mathbf{x})$ por mais que B cresça.

Ideia central

Florestas aleatórias (Breiman) *descorrelacionam* as árvores: em *cada nó*, sorteia-se um subconjunto de apenas $m < d$ covariáveis e a divisão só pode usar uma delas. Nos nós em que a variável dominante não é sorteada — uma fração $1 - m/d$ deles — a árvore é obrigada a olhar para outra coisa, e descobre estrutura que o *bagging* nunca veria. Menos ρ , menos variância.

Detalhes:

- m é o *tuning parameter*; regra empírica $m \approx d/3$ em regressão (e $m \approx \sqrt{d}$ em classificação). $m = d$ recupera o *bagging*.

- O risco é **robusto a B** : ele estabiliza e *não há superajuste* ao aumentar B — não vale a pena fazer CV em B , basta usar B “grande o bastante”.
- Conexão com a Aula 05: a taxa de convergência das florestas depende essencialmente do número de *variáveis relevantes* — elas se adaptam à *esparsidade*.

5 Boosting

Boosting também agrega árvores, mas de forma **sequencial e adaptativa**: cada nova árvore corrige os *resíduos* da combinação atual. Não reamostra; reajusta ao erro corrente.

Algoritmo (boosting de árvores para regressão)

1. Inicialize $g(\mathbf{x}) \equiv 0$ e resíduos $r_i \leftarrow Y_i, \forall i$.
2. Para $b = 1, \dots, B$:
 - (a) ajuste uma árvore *rasa* (com p folhas) a $(\mathbf{X}_1, r_1), \dots, (\mathbf{X}_n, r_n)$; chame-a g_b ;
 - (b) atualize $g(\mathbf{x}) \leftarrow g(\mathbf{x}) + \lambda g_b(\mathbf{x})$ e $r_i \leftarrow Y_i - g(\mathbf{X}_i)$.
3. Retorne $g(\mathbf{x}) = \lambda \sum_{b=1}^B g_b(\mathbf{x})$.

Ideia central: a inversão em relação ao bagging

Repare no sentido oposto de percurso. O *bagging* parte de modelos de **viés baixo e variância alta** e usa a média para derrubar a variância. O *boosting* parte de $g \equiv 0$ — **viés máximo, variância zero** — e vai *comprando viés de volta* em prestações de tamanho λ . São os dois sentidos do mesmo eixo da Aula 01, e por isso os riscos de cada um são opostos: excesso de *bagging* é inofensivo, excesso de *boosting* superajusta.

Os *tuning parameters*:

λ (*learning rate*)

fator pequeno (ex.: 0,01) que controla o quanto cada árvore contribui. λ grande $\Rightarrow$ superajuste rápido; pequeno $\Rightarrow$ aprendizado lento (precisa de B maior). λ e B são acoplados: reduzir λ pela metade pede mais ou menos o dobro de B .

p (*profundidade/folhas*)

árvores rasas ($p = 2$ a 4 ; *stumps* têm uma divisão) controlam a ordem de interações capturadas: com *stumps*, o modelo final é aditivo e não captura interação nenhuma; com profundidade 2, captura interações de duas variáveis, e assim por diante.

B (*número de árvores*)

escolhido por *early stopping*: pare quando o risco em validação parar de cair.

Atenção

Diferença crucial em relação às florestas: no *boosting*, B **grande demais superajusta**, como mostra a Figura 2. Por isso o *early stopping* é essencial — e por isso B é um hiperparâmetro no *boosting* e *não é* nas florestas. Confundir os dois casos é um dos erros mais comuns de quem está começando.

Observação 5.1. A implementação moderna mais popular é o **XGBoost** (Chen & Guestrin, 2016): *boosting* de árvores com regularização adicional, tratamento de dados faltantes e altíssimo desempenho computacional — frequentemente o melhor método “pronto” em dados tabulares.

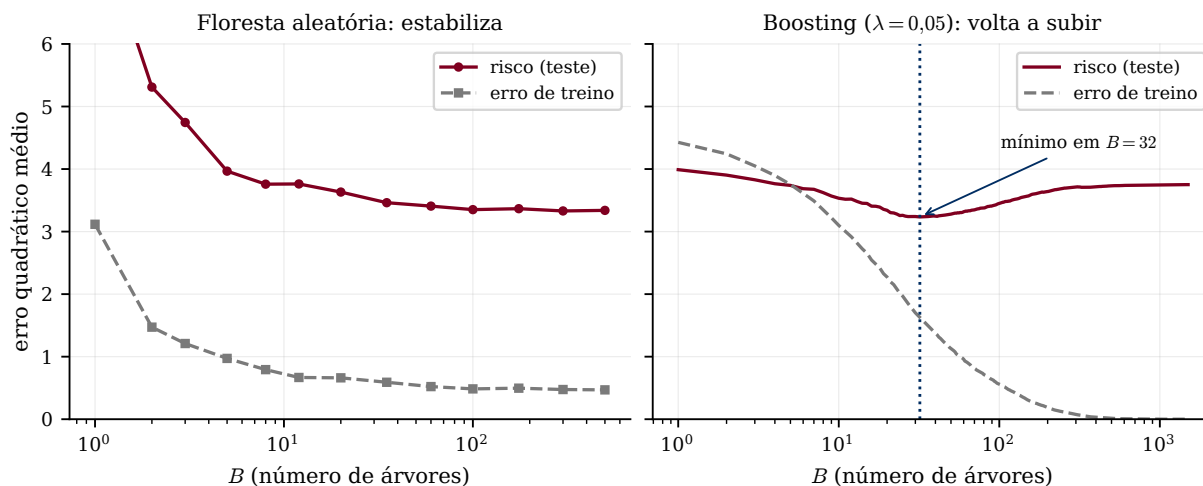


Figura 2: A diferença mais prática entre os dois métodos, medida num problema com $n = 150$, $d = 8$ e ruído alto. À esquerda, a floresta: o risco cai, estabiliza em torno de 3,34 e *fica lá* — de $B = 100$ para $B = 500$ ele varia $-0,4\%$. Você pode exagerar em B à vontade; só perde tempo de máquina. À direita, o *boosting* com $\lambda = 0,05$: o risco atinge 3,23 em $B = 32$ e depois **volta a subir**, chegando a 3,75 (+16%) em $B = 1500$, enquanto o erro de treino continua descendo rumo a zero. É o retrato do superajuste da Aula 01.

6 Prática em Python

```

1 from sklearn.tree import DecisionTreeRegressor
2 from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
3 from sklearn.model_selection import GridSearchCV
4
5 # Árvore única: cuidado com superajuste -> controle a complexidade (alpha de poda)
6 arvore = DecisionTreeRegressor(ccp_alpha=0.0) # ccp_alpha e o nosso alpha
7
8 # Floresta aleatória: B grande, m = max_features (~ d/3 em regressão)
9 rf = RandomForestRegressor(n_estimators=500, max_features=1/3,
10                           oob_score=True, random_state=0).fit(X_tr, y_tr)
11 print("risco OOB (R2):", rf.oob_score_)
12 print("importâncias:", rf.feature_importances_)
13
14 # Boosting: lambda pequeno + B por early stopping
15 gb = GradientBoostingRegressor(learning_rate=0.01, n_estimators=2000,
16                               max_depth=3, validation_fraction=0.2,
17                               n_iter_no_change=20).fit(X_tr, y_tr)

```

Observação 6.1. Três detalhes que fazem diferença nesse trecho. O `oob_score=True` entrega a estimativa OOB sem custo extra — aproveite. Os argumentos `validation_fraction` e `n_iter_no_change` são o *early stopping* do `scikit-learn`: sem eles, as 2000 árvores serão todas usadas, e a Figura 2 mostra o que isso custa. E note que árvores **não precisam** de `StandardScaler`: elas só comparam valores dentro de cada variável, então a escala é irrelevante — ao contrário do KNN da Aula 04.

O notebook Aula prática 06 e 07 compara árvore, floresta e *boosting* no `superconductivity.csv`.

Resumo

- **Árvore:** partição recursiva (1), com os dois lados da Figura 1; cresce gananciosamente (2) e poda por custo-complexidade (3) (α por CV). Interpretável, mas alta variância, e

só corta perpendicular aos eixos.

- Agregar reduz variância se os preditores são não-viesados e pouco correlacionados (Prop. 2.1); com correlação ρ , a variância trava em $\rho v(\mathbf{x})$.
- **Bagging** (4): árvores profundas em amostras bootstrap; OOB “de graça”; importância por redução de RSS (enviesada — use com cuidado).
- **Florestas**: *descorrelacionam* sorteando $m \approx d/3$ variáveis por nó; robustas a B ; adaptam-se à esparsidade.
- **Boosting**: ajuste sequencial dos resíduos, partindo do viés máximo; λ pequeno, p raso, B por *early stopping* — aqui B grande *superajusta* (Fig. 2). XGBoost é a implementação de referência.

Leitura recomendada. [AME] §4.8 (árvores, com o algoritmo de crescimento e poda), §4.9 (*bagging* e florestas aleatórias, incluindo a importância de variáveis) e §4.10 (*boosting*). [ISLP] Capítulo 8 (*Tree-Based Methods*) — §8.1 tem a versão deles da nossa Figura 1, e §8.2 compara *bagging*, florestas e *boosting* nos mesmos dados.

Para praticar. recursos/listas/Lista de exercícios 06.pdf.